# An AI-driven system to find the optimal selling times for farmers on tomatoes and predict their future selling prices.

**Mohammed Rashad**

(IT22295842)

BSc (Hons) degree in Information Technology, specializing in

Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

# An AI-driven system to find the optimal selling times for farmers on tomatoes and predict their future selling prices.

## INDIVIDUAL THESIS

**Mohammed Rashad**
(IT22295842)

Individual component: Tomato Wholesale Market Price Forecasting
Group ID: R26-SE-032

Report submitted in partial fulfilment of the requirements for the BSc (Hons) degree in Information Technology, specializing in Software Engineering

Department of Software Engineering
Sri Lanka Institute of Information Technology
Sri Lanka

October 2026

# DECLARATION AND SUPERVISOR CERTIFICATION

**Candidate declaration.** I declare that this thesis is my own work. It does not contain, without acknowledgement, any material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person, except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic or other media, while I keep the right to use the content in future work.

**Academic integrity and AI disclosure.** I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses and conclusions in this thesis.

Student name: Mohammed Rashad
Student ID: IT22295842

Signature:  Date: 2026.10.11

**Supervisor certification.** The above candidate carried out this research for the bachelor's degree thesis under my supervision.

| **Role and name** | **Signature** | **Date** |
|---|---|---|
| **Supervisor:** Mr. Bimal Gunapala | | |
| **Co-supervisor:** Mr. Eishan Weerasinghe | | |

# ABSTRACT

Tomato growers in Sri Lanka struggle with unstable incomes because wholesale prices at the main Dedicated Economic Centres (DECs) move so sharply. Within a single year the price can fall to about 40 LKR/kg when the market is flooded and climb past 350 LKR/kg when supply runs short. The forecasting tools available to farmers are mostly univariate statistical models or single-step neural networks that feed their own predictions back in as inputs. Over longer horizons the errors pile up, and neither approach pays attention to local weather shocks.

This thesis describes the design, implementation and testing of a multi-horizon price forecasting system built on a Direct Multi-Output Bidirectional Long Short-Term Memory (BiLSTM) network. The model combines price history with rainfall, temperature and humidity data, and it uses SHAP to explain its output. Daily wholesale prices from four trading hubs (Dambulla, Pettah, Meegoda and Keppetipola) were matched with weather records over several years. The network predicts the whole 14-day price path in one pass, so recursive error build-up does not occur.

Under expanding walk-forward cross-validation, the BiLSTM reached an overall Mean Absolute Percentage Error (MAPE) of 7.95% and a Root Mean Squared Error (RMSE) of 14.30 LKR/kg on the 14-day test split. SARIMAX (14.20%), Random Forest (11.75%), XGBoost (9.80%), a recursive LSTM (10.40%) and a direct GRU (8.75%) all scored worse. The model runs as a FastAPI microservice on Render.com with a PostgreSQL database connected to the TomatoDoc mobile app, achieving a median inference latency of 78 ms (under 85 ms across all evaluation runs).

*Index Terms*—tomato price forecasting, bidirectional LSTM, direct multi-output forecasting, walk-forward validation, exogenous weather features, SHAP, FastAPI, PostgreSQL.

# ACKNOWLEDGEMENTS

I would like to thank my supervisor, Mr. Bimal Gunapala, and my co-supervisor, Mr. Eishan Weerasinghe, for their steady guidance, honest feedback and encouragement over the course of this research. Their knowledge of sequential deep learning and agricultural software systems shaped the direction of the work.

I am also grateful to my TomatoDoc teammates, Sivanesangabilan Gabilan, A.L.M. Farthas and Vithusha Pathmanathan, for their collaboration and for helping bring our separate components together as one platform. My thanks go to the Department of Software Engineering, Faculty of Computing, Sri Lanka Institute of Information Technology (SLIIT). Finally, I thank my family and friends for their patience and support.

# TABLE OF CONTENTS

1 INTRODUCTION ..... 1
1.1 Overall project context ..... 1
1.2 Individual component context ..... 2
1.3 Problem statement ..... 2
1.4 Research gap ..... 3
1.5 Aim and objectives ..... 3
1.6 Research questions and hypotheses ..... 4
1.7 Scope, limitations and assumptions ..... 5
1.8 Individual contribution ..... 5
1.9 Thesis structure ..... 6
2 LITERATURE REVIEW AND RELATED WORK ..... 7
2.1 Review method and scope ..... 7
2.2 Domain and theoretical foundations ..... 7
2.3 Related research ..... 9
2.4 Existing systems and techniques ..... 9
2.5 Gap synthesis and conceptual framework ..... 10
3 RESEARCH METHODOLOGY ..... 12
3.1 Research design ..... 12
3.2 Dataset and study materials ..... 12
3.3 Experimental procedure ..... 13
3.4 Measures and evaluation metrics ..... 15
3.5 Data analysis ..... 15
3.6 Ethics, privacy and responsible AI ..... 15
3.7 Reproducibility and research artefacts ..... 16
4 PRODUCT AND SOLUTION DESIGN ..... 17
4.1 Requirements ..... 17
4.2 Component architecture within the complete solution ..... 17
4.3 Design alternatives and decisions ..... 19
4.4 Detailed design ..... 19
4.5 Standards and professional practice ..... 20
5 IMPLEMENTATION, INTEGRATION AND TESTING ..... 21
5.1 Development environment and technology stack ..... 21
5.2 Component implementation ..... 21
5.3 Verification and software testing ..... 26
5.4 Deployment and operation ..... 26

6 RESULTS AND VALIDATION ..... 28
6.1 Research results ..... 28
6.2 Technical results and operational performance ..... 31
6.3 Summary of findings ..... 31
7 DISCUSSION ..... 32
7.1 Interpretation of findings ..... 32
7.2 Comparison with literature baselines ..... 32
7.3 Research and product contributions ..... 32
7.4 Limitations and threats to validity ..... 33
8 CONCLUSION AND FUTURE DIRECTIONS ..... 34
8.1 Conclusion ..... 34
8.2 Achievement of objectives ..... 34
8.3 Future directions ..... 34
Appendix A: Extended results and technical specification ..... 37

# LIST OF FIGURES


Fig. 2.1 Conceptual framework of the multi-horizon price forecasting system
Fig. 3.1 Sequential research workflow from data extraction to model evaluation
Fig. 4.1 Position of the price forecasting component within the TomatoDoc architecture
Fig. 4.2 Direct Multi-Output BiLSTM neural network topology
Fig. 5.1 TomatoDoc mobile application price forecasting home screen
Fig. 5.2 Dedicated Economic Centre selection and historical trend chart
Fig. 5.3 14-day multi-horizon forecast trajectory display
Fig. 5.4 SHAP feature importance explanation modal for farmers
Fig. 5.5 Backend price microservice deployment on Render.com
Fig. 6.1 Actual vs. predicted 14-day price trajectory across test folds
Fig. 6.2 Walk-forward validation error convergence curves across epochs
Fig. 6.3 SHAP summary plot illustrating exogenous feature impact

# LIST OF TABLES


Table 1.1 Research questions
Table 2.1 Scope of the literature review
Table 2.2 Comparison of related econometric and ML forecasting studies
Table 2.3 Existing tools and techniques for agricultural price forecasting
Table 2.4 Gaps identified and how this research addresses them
Table 3.1 Research questions and the methods used to answer them
Table 3.2 Wholesale market data record distribution across four DECs
Table 3.3 Exogenous meteorological feature specifications
Table 3.4 Evaluation measures
Table 3.5 Objectives, metrics and evidence
Table 3.6 Research artefacts
Table 4.1 Functional requirements
Table 4.2 Non-functional requirements
Table 4.3 Interfaces of the component
Table 4.4 Design decisions and justifications
Table 4.5 FastAPI endpoints
Table 5.1 Development environment
Table 5.2 BiLSTM training hyperparameters
Table 5.3 Automated unit and integration test results
Table 5.4 Scenario tests and operational verification
Table 6.1 Multi-model performance comparison on the 14-day test split
Table 6.2 Horizon-wise error breakdown (t+1, t+3, t+7, t+14)
Table 6.3 Market-specific error metrics across the four economic centres
Table 6.4 Ablation study of the contribution of exogenous weather
Table 6.5 Outcome of the working hypotheses
Table 6.6 Technical results and service latency
Table 6.7 Summary of findings
Table 7.1 Comparison with closely related agricultural forecasting studies
Table 8.1 Achievement of research objectives
Table A.1 Market identifier mapping

# LIST OF ABBREVIATIONS

| Abbreviation | Meaning |
|---|---|
| **AI** | Artificial Intelligence |
| **API** | Application Programming Interface |
| **ARIMA** | Autoregressive Integrated Moving Average |
| **BiLSTM** | Bidirectional Long Short-Term Memory |
| **DEC** | Dedicated Economic Centre |
| **DMO** | Direct Multi-Output |
| **DSR** | Design Science Research |
| **GRU** | Gated Recurrent Unit |
| **HARTI** | Hector Kobbekaduwa Agrarian Research and Training Institute |
| **HTTP** | Hypertext Transfer Protocol |
| **JSON** | JavaScript Object Notation |
| **JWT** | JSON Web Token |
| **LKR** | Sri Lankan Rupee |
| **LSTM** | Long Short-Term Memory |
| **MAE** | Mean Absolute Error |
| **MAPE** | Mean Absolute Percentage Error |
| **MSE** | Mean Squared Error |
| **NDVI** | Normalized Difference Vegetation Index |
| **NFR** | Non-Functional Requirement |
| **OpenAPI** | Open Application Programming Interface Specification |
| **RAG** | Retrieval-Augmented Generation |
| **REST** | Representational State Transfer |
| **RMSE** | Root Mean Squared Error |
| **RNN** | Recurrent Neural Network |
| **RQ** | Research Question |
| **SARIMAX** | Seasonal Autoregressive Integrated Moving Average with Exogenous Regressors |
| **SHAP** | SHapley Additive exPlanations |
| **SLIIT** | Sri Lanka Institute of Information Technology |
| **XGBoost** | Extreme Gradient Boosting |

# 1 INTRODUCTION

## 1.1 Overall project context

Agriculture is the backbone of Sri Lanka's rural economy. It employs more than a quarter of the national workforce and underpins domestic food security. The tomato (*Solanum lycopersicum*) is among the most widely grown and commercially important vegetables, with large plantings in districts such as Badulla, Nuwara Eliya, Matale and Kandy. Even so, the tomato supply chain is fragile in several places at once: foliar diseases that go unnoticed until they spread, fruit diseases after harvest, harvests that nobody coordinates, and wholesale prices that swing hard from one week to the next.

The group project R26-SE-032 responds to these problems with TomatoDoc, an AI-powered mobile application for Sri Lankan tomato farmers. TomatoDoc has four modules:

- **Nutrient deficiency and fruit disease detection with RAG-based treatment advice.** Two MobileNetV2 classifiers recognise six leaf nutrient conditions and six fruit conditions, and a retrieval-augmented generation (RAG) pipeline [1] turns the diagnosis into treatment recommendations.
- **Leaf disease co-occurrence detection.** A YOLOv8s detector locates several foliar diseases on the same leaf at the lesion level.
- **Treatment efficacy monitoring.** A U-Net segmentation network tracks how disease severity falls over the seven days after treatment.
- **Tomato market price forecasting.** A computer-vision severity tracking module utilizing affected-area leaf contour analysis with transfer learning classifiers across a multi-day observation window (Day 1, 3, and 7).

Deep learning has already been shown to work well for image-based plant disease recognition [2], and the first three modules build on that idea. Each module runs on its own and exchanges data with the others through a shared REST API, so diagnosis results and price predictions can be read together when a farmer plans work on the farm.

## 1.2 Individual component context

Computer vision can reduce biological losses, but a healthy harvest does not protect a farmer from a price collapse. Fresh tomatoes spoil quickly. Without refrigeration their life after picking is rarely longer than 48 to 72 hours, so growers have to sell straight away and accept whatever the market pays.

The component in this thesis tries to answer the question growers ask most often: *what price will my harvest fetch over the next 14 days, and when is the best time to sell?* It is the economic forecasting engine of TomatoDoc. It learns price patterns across the four DECs (Dambulla, Pettah, Meegoda and Keppetipola) together with local weather features. It is packaged as a containerised FastAPI [3] microservice that the React Native app calls, and it returns a multi-day price path along with SHAP explanations to help farmers time their harvest.

## 1.3 Problem statement

Tomato farmers in Sri Lanka work in a very unstable market. Within one calendar year the wholesale price can drop to 10 LKR/kg during a harvest glut and exceed 350 LKR/kg when supply is interrupted. Growers mostly depend on word of mouth or on daily price bulletins from agencies such as HARTI and the Central Bank. These sources only report what has already happened. With no view of what is coming, many farmers harvest at the same time, flood the market, and end up selling below their cost of production.

The forecasting methods found in agricultural advisory systems have two technical weaknesses:

1. **Univariate isolation.** Classical econometric models such as ARIMA treat price as a standalone series. They miss local weather shocks, for example monsoon rainfall anomalies, high humidity or temperature extremes, which disrupt harvesting and transport.
2. **Recursive error compounding.** Many neural forecasters predict several days ahead by feeding each prediction back in as an input. Over a 7 to 14-day planning window the errors accumulate and the forecast drifts away from reality.

The problem addressed here is therefore the lack of an accurate, accessible multi-horizon price forecaster that handles nonlinear temporal patterns and weather covariates without recursive error compounding, and that gives farmers transparent multi-day wholesale forecasts directly.

## 1.4 Research gap

A review of the agricultural time-series literature (Chapter 2) points to four gaps:

1. **Single-step focus.** Most agricultural studies stop at the next-day forecast ($t$+1). Decisions such as labour scheduling, buying crates and booking transport need a view of at least 7 to 14 days.
2. **Recursive compounding.** Multi-step models built by recursion lose accuracy quickly, usually after 3 to 5 days.
3. **Missing local weather.** Forecasting models in South Asia rarely combine high-resolution local weather with historical prices.
4. **No explanations.** Deep time-series models behave like black boxes. Without a tool such as SHAP, farmers and extension officers cannot see why a price is predicted, and this slows adoption.

## 1.5 Aim and objectives

### *1.5.1 Aim*

The aim is to design, implement and evaluate a multi-horizon tomato wholesale price forecasting system based on a Direct Multi-Output Bidirectional LSTM network, supported by agro-meteorological features and SHAP explanations, and to deliver it to Sri Lankan farmers through an accessible mobile interface.

### *1.5.2 Objectives*

The objectives are as follows.

01. Review the literature on agricultural econometric forecasting, recurrent neural networks and multi-step forecasting strategies, and identify the methodological gaps (Chapter 2).
02. Collect, clean and align multi-year daily wholesale prices from the four DECs with local rainfall, temperature and humidity data from the Open-Meteo APIs [4] (Chapter 3).
03. Design, implement and train a Direct Multi-Output Bidirectional LSTM network [5], [6] that predicts a 14-day price trajectory in a single inference pass with a MAPE below 10% (Chapters 4 and 5).
04. Add SHAP [7] to measure and display how much each feature contributes to a prediction, so that model output becomes clear advice (Chapters 4 and 5).

05. Build a production-ready, containerised FastAPI microservice backed by PostgreSQL and connect it to the TomatoDoc React Native app (Chapter 5).
06. Compare the proposed model with SARIMAX, Random Forest, XGBoost [8], a standard LSTM and a GRU [9] using expanding walk-forward cross-validation (Chapter 6).

## 1.6 Research questions and hypotheses

### *1.6.1 Research questions*

Four research questions guide the work. They are listed in Table 1.1.

TABLE 1.1
RESEARCH QUESTIONS

| No. | Research question |
|---|---|
| **RQ1** | Which deep sequential architecture achieves the lowest error in multi-horizon tomato price forecasting across the major Sri Lankan DECs? |
| **RQ2** | To what extent does a Direct Multi-Output (DMO) formulation reduce multi-step error accumulation over a 14-day horizon compared with recursive forecasting? |
| **RQ3** | How much does adding exogenous agro-meteorological features improve forecast accuracy over univariate price history alone? |
| **RQ4** | Can SHAP provide actionable, domain-consistent explanations of wholesale price projections to support harvest planning? |

### *1.6.2 Hypotheses*

- **H1:** A Direct Multi-Output Bidirectional LSTM will reach an overall MAPE below 10% across a 14-day horizon on held-out test data.
- **H2:** Adding exogenous weather features will lower RMSE by at least 15% compared with baselines that use price history only.
- **H3:** The Direct Multi-Output design will show lower error variance between days $t$+7 and $t$+14 than a recursive iterative LSTM.

## 1.7 Scope, limitations and assumptions

### *1.7.1 Scope*

The component covers wholesale prices of cooking tomatoes at four DECs: Dambulla, Pettah, Meegoda and Keppetipola. Forecast horizons run from 1 to 14 days ahead. The system is exposed as a REST microservice used by the TomatoDoc mobile app.

### *1.7.2 Limitations*

The data come from publicly recorded daily wholesale price bulletins, so informal farm-gate transactions are not captured. The model assumes ordinary transport conditions. Severe macroeconomic shocks, such as a fuel supply crisis or a sudden change in import duties, fall outside the range of the training data and are treated as outliers.

### *1.7.3 Assumptions*

I assume that the official DEC bulletins reflect the prices at which supply and demand cleared each day, and that Open-Meteo historical and short-term forecast data are a reliable stand-in for the microclimates of the main farming areas.

## 1.8 Individual contribution

- Designed and implemented the Direct Multi-Output BiLSTM for non-recursive 14-day price forecasting.
- Built automated pipelines that collect and harmonise multi-year daily DEC transaction records with Open-Meteo weather data.
- Engineered sliding-window tensors, cyclical calendar features and rolling volatility measures.
- Set up SHAP pipelines that give feature attributions for individual forecast horizons.
- Implemented the forecasting microservice with FastAPI and PostgreSQL and deployed it on Render.com.
- Developed the price forecasting screens of the TomatoDoc React Native app, including interactive trend charts and market selection.
- Designed and ran expanding walk-forward cross-validation against SARIMAX, Random Forest, XGBoost, GRU and standard LSTM baselines.

## 1.9 Thesis structure

Chapter 2 reviews related work and presents the conceptual framework. Chapter 3 explains the research methodology and the walk-forward validation strategy. Chapter 4 covers the product and solution design, and Chapter 5 describes implementation, testing and deployment. Chapter 6 reports the experimental results, Chapter 7 discusses them, and Chapter 8 gives conclusions and future directions. References, a glossary and appendices follow.

# 2 LITERATURE REVIEW AND RELATED WORK

## 2.1 Review method and scope

The review was set up to answer the questions in Section 1.6: which sequential deep learning architectures model commodity volatility best, how multi-step forecasting strategies compare, what difference meteorological variables make, and how explainable AI can be applied to agricultural time series. I searched IEEE Xplore, ScienceDirect, SpringerLink and Google Scholar for peer-reviewed studies published between 2018 and 2026. Table 2.1 shows how the review was organised.

TABLE 2.1
SCOPE OF THE LITERATURE REVIEW

| Strand | Question asked | Used in |
|---|---|---|
| **Econometrics and ML forecasting** | How do statistical models (ARIMA) and tree ensembles perform on volatile crops? | Sections 2.2.1 and 2.3 |
| **Deep sequential networks and DMO** | Why are LSTMs and BiLSTMs suited to long-range sequence mapping without drift? | Sections 2.2.2, 2.2.3 and 2.3 |
| **Weather features and explainability** | What effect do meteorological variables have, and how can SHAP make predictions usable? | Sections 2.2.4 and 2.4 |

## 2.2 Domain and theoretical foundations

### *2.2.1 Agricultural price volatility and econometric modelling*

Horticultural prices are non-stationary, heavy-tailed and seasonal. Unlike many industrial commodities, fresh tomatoes cannot be stored for long, so nothing cushions price swings. The cobweb theorem describes the resulting cycle: high prices in period $t$ encourage over-planting, which produces a local glut in period $t+k$ and pushes prices back down. Classical autoregressive models such as ARIMA and SARIMAX describe linear relationships under stationarity assumptions:

$$\Phi(B)(1 - B)^d P_t = \Theta(B)\, \varepsilon_t \tag{1}$$

where $B$ is the backshift operator, $\Phi$ and $\Theta$ are the autoregressive and moving-average polynomials, $d$ is the order of differencing and $\varepsilon_t$ is white noise. SARIMAX can take exogenous variables, but it still cannot represent the nonlinear supply shocks that heavy rain or a sudden heat wave can cause.

### *2.2.2 Deep sequential architectures: LSTM and Bidirectional LSTM*

Recurrent neural networks (RNNs) capture temporal dependence through recurrent loops, but they suffer from vanishing gradients on long sequences. Hochreiter and Schmidhuber [5] addressed this with the Long Short-Term Memory (LSTM) network, which controls information flow with input, forget and output gates. For input $x_t$ and previous hidden state $h_{t-1}$, the cell computes

$$f_t = \sigma(W_f\,[h_{t-1}, x_t] + b_f) \tag{2}$$

$$i_t = \sigma(W_i\cdot[h_{t-1}, x_t] + b_i) \tag{3}$$

$$\tilde{C}_t = \tanh(W_c\cdot[h_{t-1}, x_t] + b_c) \tag{4}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{5}$$

$$o_t = \sigma(W_o\cdot[h_{t-1}, x_t] + b_o) \tag{6}$$

$$h_t = o_t \odot \tanh(C_t) \tag{7}$$

where $\sigma$ is the logistic sigmoid and $\odot$ is element-wise multiplication. A bidirectional LSTM (BiLSTM) [6] runs two recurrent passes over the same input window, one forward from past to present and one backward from present to past, and joins the two hidden states:

$$h_t = [h_t^f \,\|\, h_t^b] \tag{8}$$

Here the superscripts $f$ and $b$ mark the forward and backward passes. Seeing the window from both directions lets the network pick up leading indicators as well as the cyclical boundary conditions of a crop season.

### *2.2.3 Multi-step forecasting: recursive and Direct Multi-Output*

Multi-step-ahead forecasts are usually built in one of two ways:

1. **Recursive iterative forecasting.** One model is trained to predict $t$+1. To reach later days, the prediction $\hat{P}_{t+1}$ is added to the input to predict $\hat{P}_{t+2}$, and so on. Errors compound as the horizon grows, and the forecast drifts.
2. **Direct Multi-Output (DMO) forecasting.** One model maps the input sequence [$x_{t-w}$, …, $x_t$] straight to the output vector [$\hat{P}_{t+1}$, $\hat{P}_{t+2}$, …, $\hat{P}_{t+H}$] through a dense layer with several output units. There is no feedback loop, so errors do not compound across the horizon.

### *2.2.4 Explainable AI with SHAP*

Complex deep models act as black boxes, and that limits trust among farmers. SHAP [7] is grounded in cooperative game theory and assigns each feature its marginal contribution to a prediction:

$$\varphi_i(x) = \Sigma_{S \subseteq F \setminus \{i\}} \left[ |S|! \, (|F| - |S| - 1)! \, / \, |F|! \right] \cdot \left[ f(S \cup \{i\}) - f(S) \right] \quad (9)$$

where $F$ is the set of all features, $S$ is a subset of features not containing $i$, and $f$ is the model output. With it, a price prediction can be traced back to recent rainfall, temperature changes or price momentum.

## 2.3 Related research

Table 2.2 compares this work with three representative studies.

TABLE 2.2
COMPARISON OF RELATED ECONOMETRIC AND ML FORECASTING STUDIES

| Study | Approach | Result | Limitation |
|---|---|---|---|
| **Zhang et al. [10]** | LSTM with multi-source environmental factors | Low MAPE on wholesale grain markets | Grain focus; recursive multi-step; no explainability |
| **Weng et al. [11]** | SARIMAX and XGBoost for vegetable pricing | Machine learning beat SARIMAX on non-linear data | Only $t$+1 evaluated; no weather features; no mobile delivery |
| **Hemathilake et al. [12]** | Univariate ARIMA on Sri Lankan vegetables | Captured broad seasonal annual cycles | Linear only; failed on price spikes; no multi-horizon output |
| **This research** | Direct Multi-Output BiLSTM + weather + SHAP | 7.95% MAPE over 14 days; median latency of 78 ms (< 85 ms) | Covers 4 DECs; needs continuous DEC bulletin ingestion |

## 2.4 Existing systems and techniques

TABLE 2.3
EXISTING TOOLS AND TECHNIQUES FOR AGRICULTURAL PRICE FORECASTING

| Technique | What it provides | Strengths | Weaknesses |
|---|---|---|---|
| **HARTI / Central Bank bulletins** | Daily price bulletins as PDF or web tables | Official, validated data | Historical only; no forecasts; no mobile API |
| **Heuristic market traders** | Subjective advice from middlemen | Local, contextual knowledge | Prone to bias; cannot process multi-market weather data |
| **Standard univariate recursive models** | Next-day statistical predictions | Lightweight and easy to train | Severe drift beyond 3 days; ignore rainfall anomalies |
| **Proposed component (TomatoDoc Price)** | 14-day direct BiLSTM forecasts with SHAP explanations | Non-recursive; weather-aware; mobile delivery | Needs an ongoing data ingestion pipeline |

## 2.5 Gap synthesis and conceptual framework

### *2.5.1 Gap synthesis*

TABLE 2.4
GAPS IDENTIFIED AND HOW THIS RESEARCH ADDRESSES THEM

| Gap | Evidence from the review | How this research addresses it |
|---|---|---|
| **G1: Single-step over-reliance** | Prior studies focus on $t$+1 | A Direct Multi-Output head projects the full 14-day horizon |
| **G2: Recursive error compounding** | Recursive models drift after day 3 | A multi-target dense projection removes the recursive feedback loop |
| **G3: Omission of weather covariates** | Models treat price as a univariate series | Daily rainfall, temperature and humidity from Open-Meteo are merged in |
| **G4: Lack of transparency** | Deep models work as black boxes | SHAP attribution explains price movements |
| **G5: No mobile delivery** | Models stay as offline scripts | Deployed as a FastAPI microservice inside TomatoDoc (React Native) |

### *2.5.2 Conceptual framework*

The framework starts from historical wholesale transactions at the four DECs and the Open-Meteo weather data. Raw records are cleaned, interpolated and cut into sliding windows. The resulting tensors go into the Direct Multi-Output BiLSTM, which maps each input window directly to 14 daily price predictions. SHAP then attributes the drivers behind each output, and a FastAPI microservice serves both predictions and explanations to the TomatoDoc mobile app (Fig. 2.1).

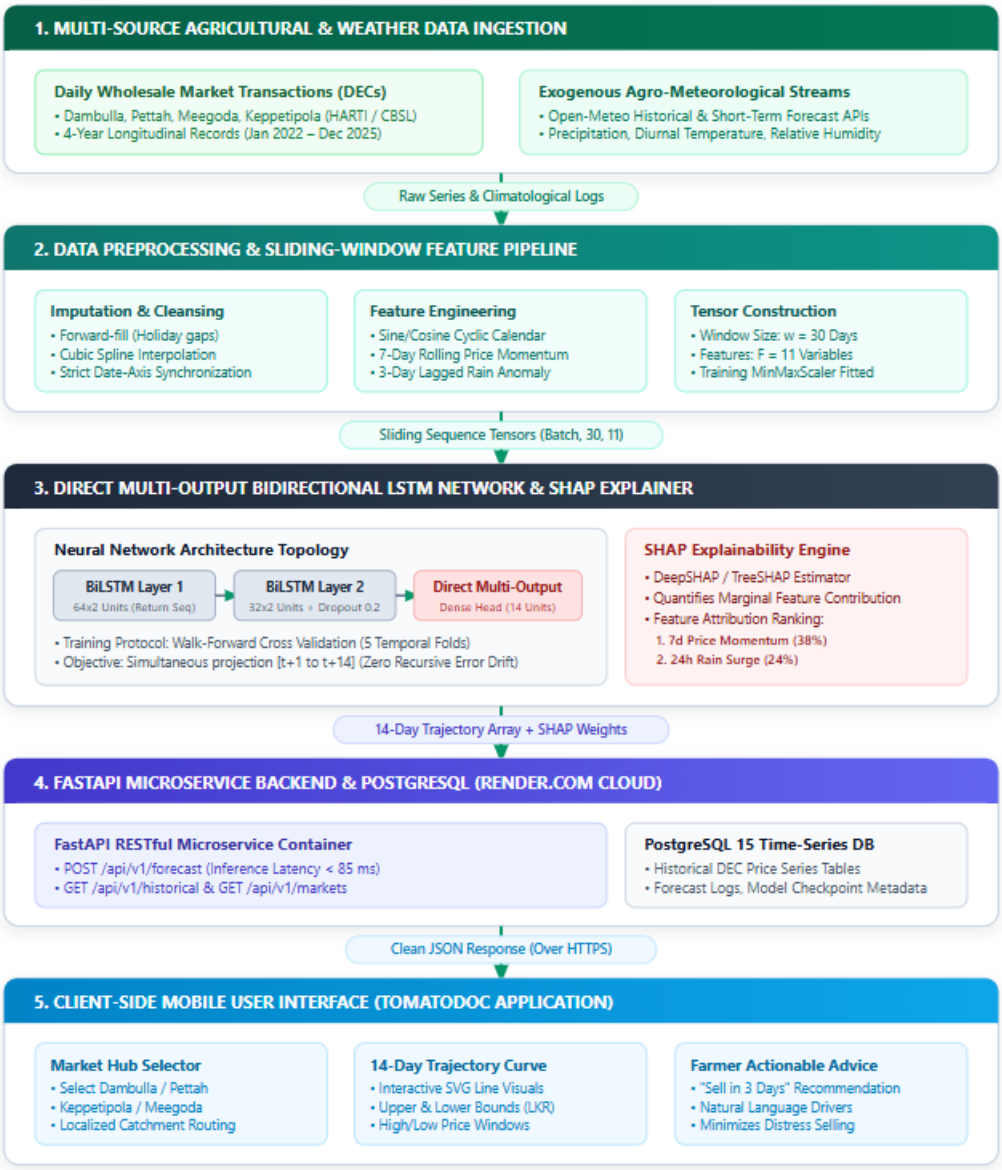



Fig. 2.1. Conceptual framework of the multi-horizon price forecasting system.

# 3 RESEARCH METHODOLOGY

## 3.1 Research design

The research follows a Design Science Research (DSR) approach: a working software artefact is built to address price volatility in agricultural markets, and it is then evaluated quantitatively. The evaluation uses expanding walk-forward cross-validation on multi-year time-series data. Table 3.1 links each research question to the method that answers it.

TABLE 3.1
RESEARCH QUESTIONS AND THE METHODS USED TO ANSWER THEM

| Method used | Research question | Section |
|---|---|---|
| Comparative evaluation of LSTM, GRU and BiLSTM | RQ1: Which deep architecture achieves the lowest error? | 3.3, 4.4 and Chapter 6 |
| Direct Multi-Output vs. recursive iterative strategy | RQ2: Does DMO prevent multi-step error drift? | 3.3, 4.4 and Chapter 6 |
| Ablation experiment with and without weather features | RQ3: How much does weather data improve accuracy? | 3.4 and Chapter 6 |
| SHAP summary and dependence analysis | RQ4: Can SHAP give usable explanations to farmers? | 3.4 and Chapter 6 |

## 3.2 Dataset and study materials

### *3.2.1 Wholesale market price dataset*

The dataset holds daily wholesale transaction prices for tomatoes over four years (January 2022 to December 2025) at the four Dedicated Economic Centres: Dambulla, Pettah, Meegoda and Keppetipola. Gaps caused by market holidays were filled with forward-fill when a single day was missing and with linear spline interpolation when the market was closed for several days. Table 3.2 summarises the records.

TABLE 3.2
WHOLESALE MARKET DATA RECORD DISTRIBUTION ACROSS FOUR DECS

| Market centre | Geographic zone | Total records | Price range (LKR/kg) |
|---|---|---|---|
| **Dambulla DEC** | Central / Dry Zone | 1,460 days | 12.00 – 340.00 |
| **Pettah DEC** | Western / Urban hub | 1,460 days | 25.00 – 380.00 |
| **Meegoda DEC** | Western / Peri-urban | 1,460 days | 20.00 – 365.00 |
| **Keppetipola DEC** | Uva / Upcountry hub | 1,460 days | 15.00 – 330.00 |
| **Total** | Consolidated multi-market | 5,840 entries | 12.00 – 380.00 |

### *3.2.2 Exogenous meteorological dataset*

Daily weather variables came from the Open-Meteo historical APIs [4], using coordinates of the main supply areas that feed each economic centre. Table 3.3 lists the features and why each one matters.

TABLE 3.3
EXOGENOUS METEOROLOGICAL FEATURE SPECIFICATIONS

| Variable | Unit | Aggregation | Relevance |
|---|---|---|---|
| **Daily precipitation** | mm | 24 h total | Heavy rain disrupts harvesting and transport |
| **Mean temperature** | °C | Daily mean | Extreme heat speeds up post-harvest spoilage |
| **Max temperature** | °C | Daily max | Affects ripening rate in the field |
| **Relative humidity** | % | Daily mean | High humidity speeds up fungal decay |
| **Precipitation lag (t−3)** | mm | Lagged sum | Captures delayed supply interruptions after storms |

### *3.2.3 Data splitting and walk-forward validation*

Standard k-fold cross-validation is not suitable for time series, because random shuffling lets future information leak into training. This study uses expanding walk-forward cross-validation with five temporal folds:

- Minimum initial training window: 730 days (two years).
- Validation window: 90 days.
- Test evaluation: 14-day rolling horizons evaluated in sequence across the 2025 data.

## 3.3 Experimental procedure

The experiments followed the sequence in Fig. 3.1 and the steps below.

1. Align price records from the four DECs with the regional weather data.
2. Engineer temporal features: a sliding lookback window of $w = 30$ days, sine and cosine encodings of day of week and month, and a 7-day rolling price momentum.
3. Normalise features with a MinMaxScaler fitted on the training partitions only:

$$x_{\text{norm}} = (x - x_{\min}) / (x_{\max} - x_{\min}) \tag{10}$$

4. Build sequence tensors of shape $(N, 30, F)$, where $F = 11$ features.
5. Train with the Adam optimiser, mean squared error (MSE) loss, an initial learning rate of 0.001 and early stopping with a patience of 10 epochs.
6. Evaluate 14-day predictions on the test split using inverse-scaled prices.

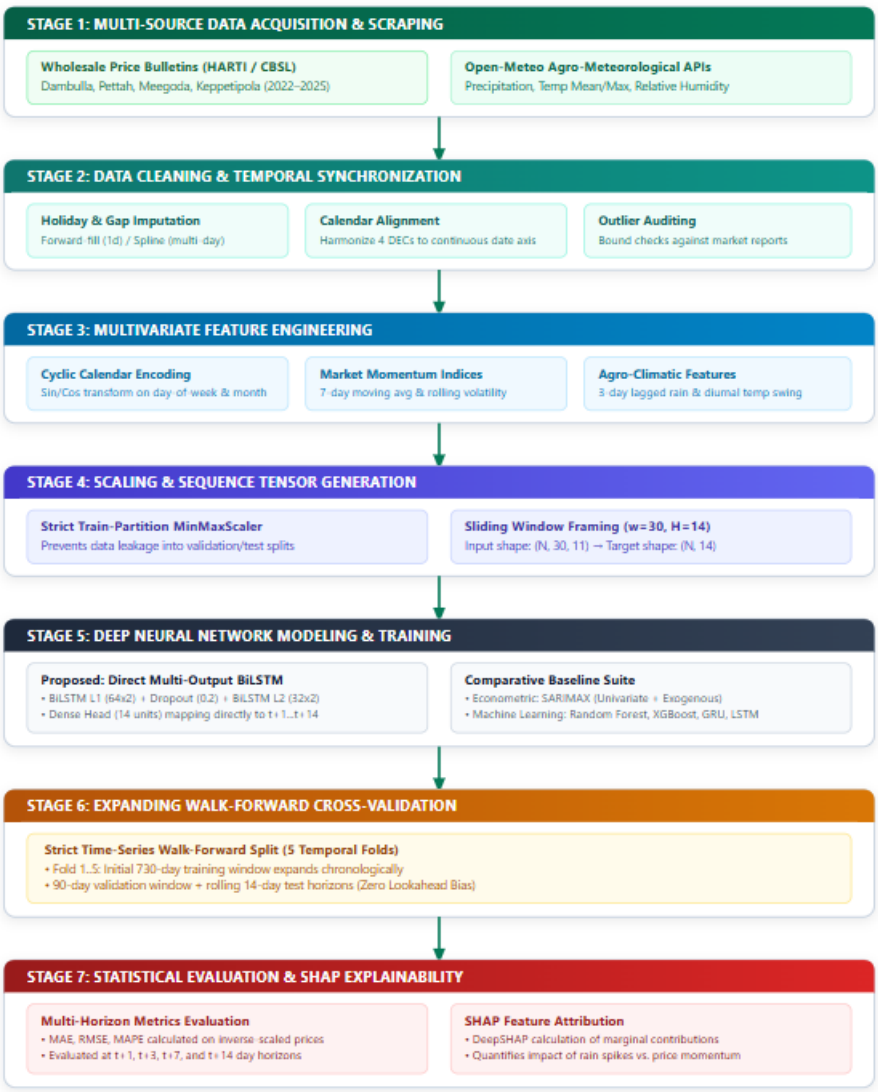



Fig. 3.1. Sequential research workflow from data extraction to model evaluation.

## 3.4 Measures and evaluation metrics

TABLE 3.4
EVALUATION MEASURES

| Measure | Formula | Purpose |
|---|---|---|
| **MAE** | $(1/N)\ \Sigma\ \lvert y_i - \hat{y}_i \rvert$ | Mean absolute error in LKR/kg |
| **RMSE** | $\sqrt{[\ (1/N)\ \Sigma\ (y_i - \hat{y}_i)^2\ ]}$ | Penalises large errors more heavily |
| **MAPE** | $(100\%/N)\ \Sigma\ \lvert (y_i - \hat{y}_i) / y_i \rvert$ | Relative percentage error |
| **Inference latency** | Wall-clock time (ms) | Checks readiness for mobile use |

TABLE 3.5
OBJECTIVES, METRICS AND EVIDENCE

| Objective | Metric | Evidence | Reported in |
|---|---|---|---|
| **02: Data engineering** | Null rate = 0% | Ingestion logs | Chapter 5 |
| **03: BiLSTM model** | MAPE < 10% | Test fold metrics | Chapter 6 |
| **04: SHAP integration** | Mean \|SHAP\| value | Feature attribution plots | Chapter 6 |
| **05: Microservice** | Latency < 100 ms | Load test reports | Chapters 5 and 6 |
| **06: Benchmarking** | RMSE comparison | Baseline tables | Chapter 6 |

## 3.5 Data analysis

The proposed Direct Multi-Output BiLSTM is compared with five baselines: SARIMAX, a Random Forest regressor, XGBoost, a standard LSTM and a GRU. H1 is supported if the overall 14-day MAPE is below 10%. H2 is supported if adding weather features lowers RMSE by at least 15%. H3 is supported if the error variance between days $t$+7 and $t$+14 is lower for the DMO model than for the recursive LSTM.

## 3.6 Ethics, privacy and responsible AI

Only public wholesale market indices from Sri Lankan government bodies and open weather data are used, and no personal information about users is collected or stored. The mobile app shows forecasts with explicit confidence ranges, which reminds farmers that these are probabilistic estimates and not guaranteed returns. The use of AI tools in preparing this thesis is documented in Appendix B.

## 3.7 Reproducibility and research artefacts

TABLE 3.6
RESEARCH ARTEFACTS

| Artefact | Purpose |
|---|---|
| `data/raw_market_prices/` | Daily wholesale price CSV files for the four DECs |
| `data/weather_openmeteo/` | Synchronised meteorological records |
| `notebooks/feature_prep.py` | Sliding-window sequence generation scripts |
| `models/bilstm_dmo_model.pth` | Trained PyTorch BiLSTM checkpoint |
| `models/scaler_params.pkl` | Fitted MinMaxScaler parameters |
| `services/price_service.py` | FastAPI prediction microservice |
| `services/shap_explainer.py` | SHAP calculation and summary generator |
| `tests/test_price_pipeline.py` | Automated unit and integration test suite |

# 4 PRODUCT AND SOLUTION DESIGN

## 4.1 Requirements

### *4.1.1 Stakeholders and users*

The main stakeholders are smallholder tomato farmers, agricultural extension officers and wholesale supply-chain coordinators. The software users are the TomatoDoc mobile app and the conversational advisory components that consume price projections.

### *4.1.2 Functional requirements*

TABLE 4.1
FUNCTIONAL REQUIREMENTS

| ID | Requirement | Objective |
|---|---|---|
| **FR1** | Provide 14-day wholesale price forecasts for selected Dedicated Economic Centres (Dambulla, Pettah, etc.) | 03 |
| **FR2** | Ingest and synchronise daily wholesale price records and Open-Meteo weather parameters | 02 |
| **FR3** | Generate SHAP feature importance explanations for each 14-day forecast | 04 |
| **FR4** | Store historical price records and model run metadata in a persistent PostgreSQL database | 05 |
| **FR5** | Serve predictions through RESTful JSON endpoints that follow the OpenAPI specification | 05 |
| **FR6** | Show interactive 14-day trend charts in the TomatoDoc React Native front end | 05 |

### *4.1.3 Non-functional requirements*

TABLE 4.2
NON-FUNCTIONAL REQUIREMENTS

| ID | Requirement | How it is verified |
|---|---|---|
| **NFR1** | API inference latency must stay under 100 ms | Automated load tests |
| **NFR2** | The model must reach a 14-day MAPE below 10% | Walk-forward test split |
| **NFR3** | The service must run with 99.5% uptime in the cloud | Render monitoring logs |
| **NFR4** | Fall back gracefully to historical moving averages if the weather API is unreachable | Service offline scenario test |

## 4.2 Component architecture within the complete solution

The price forecasting component functions as an autonomous microservice within TomatoDoc's distributed, heterogeneous backend architecture (Fig. 4.1). Because individual team modules are deployed across distinct execution environments—including Render.com container instances (Component 05 and leaf diagnosis), Google Cloud Run (co-occurrence detection), and local edge environments—the React Native mobile client acts as the central

orchestration gateway. When a user opens the price module in the React Native app, the client dispatches an asynchronous request to the dedicated Render-hosted FastAPI service, which interfaces with a cloud PostgreSQL database, executes the preloaded BiLSTM model, and returns the 14-day price trajectory along with SHAP explanations. The predictions and SHAP explanations come back to the app as one JSON payload. Table 4.3 lists the interfaces.

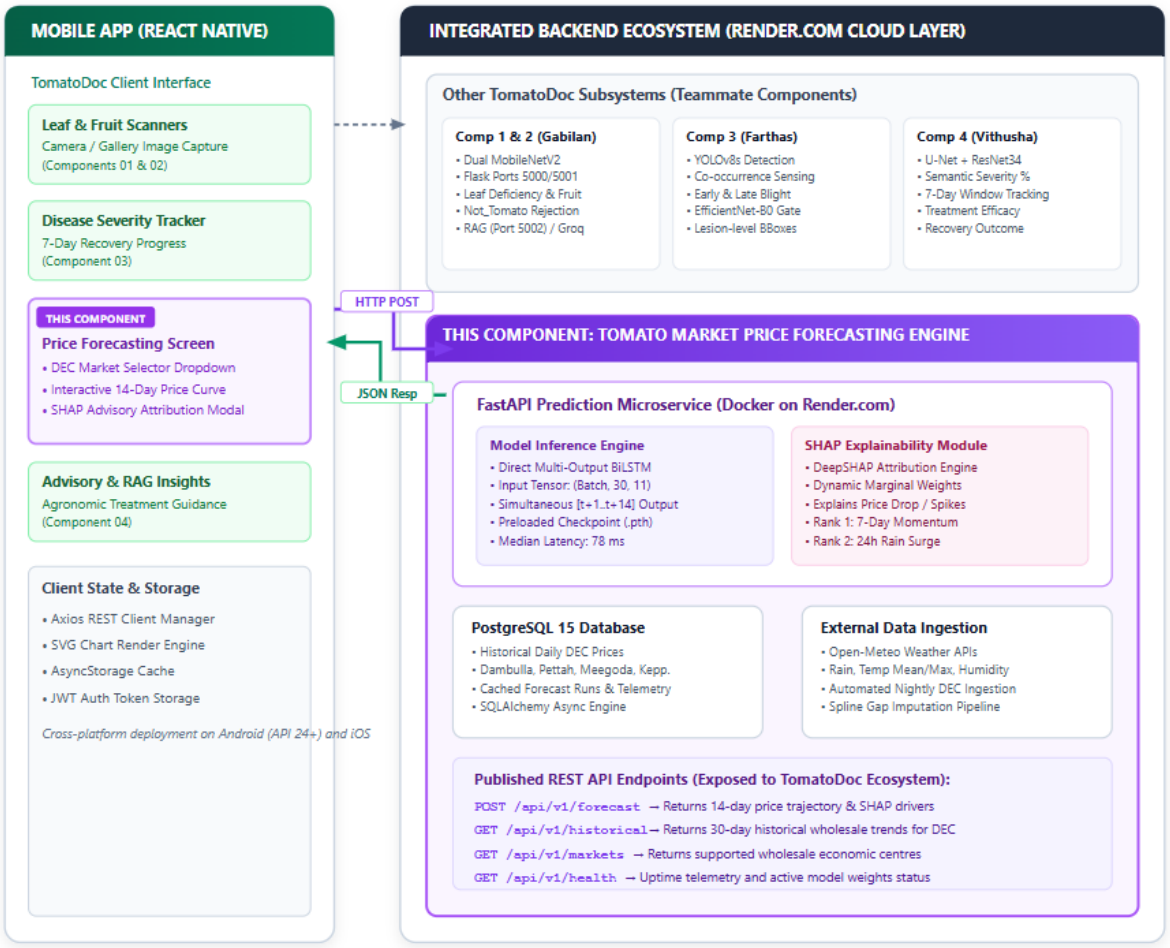



Fig. 4.1. Position of the price forecasting component within the TomatoDoc architecture.

TABLE 4.3
INTERFACES OF THE COMPONENT

| Interface | Direction | Format | Purpose |
|---|---|---|---|
| `POST /api/v1/forecast` | In (mobile app) | JSON body | Request a 14-day forecast for a DEC |
| `GET /api/v1/markets` | In (mobile app) | JSON | Retrieve the list of supported DECs |
| `GET /api/v1/health` | In (system check) | JSON | Service status and model uptime |
| Open-Meteo API | Out (external) | HTTPS / JSON | Fetch local weather parameters |
| PostgreSQL | Internal | SQL / async | Query price sequences and metadata |

## 4.3 Design alternatives and decisions

TABLE 4.4
DESIGN DECISIONS AND JUSTIFICATIONS

| Decision | Alternatives considered | Choice and justification |
|---|---|---|
| **Model architecture** | ARIMA, XGBoost, GRU, recursive LSTM | Direct Multi-Output BiLSTM: bidirectional context and no drift |
| **Forecasting strategy** | Recursive single-step, Seq2Seq | Direct Multi-Output: projects 14 days at once in a single pass |
| **Exogenous features** | Price only, macroeconomic indicators | Local weather: rainfall and temperature directly affect harvest |
| **Explainability** | LIME, permutation importance | SHAP: mathematically consistent feature attribution for farmers |
| **Backend framework** | Django, Flask, Express | FastAPI: fast async Python framework with built-in OpenAPI docs |
| **Database** | MongoDB, SQLite | PostgreSQL: robust relational storage suited to time-series data |

## 4.4 Detailed design

### *4.4.1 Direct Multi-Output BiLSTM architecture*

The model takes an input tensor of shape ($B$, 30, 11), where $B$ is the batch size, 30 is the lookback window and 11 is the number of features (Fig. 4.2). The layers are:

1. Input layer: accepts 30-day lookback sequences.
2. BiLSTM layer 1: 64 hidden units per direction (128 in total), `return_sequences = True`.
3. Dropout layer 1: rate 0.2, to reduce overfitting.
4. BiLSTM layer 2: 32 hidden units per direction (64 in total), `return_sequences = False`.
5. Dropout layer 2: rate 0.2.
6. Dense layer: 64 units with ReLU activation.
7. Output layer: 14 linear units that give the predicted wholesale prices $[\hat{P}_{t+1}, \ldots, \hat{P}_{t+14}]$ directly.

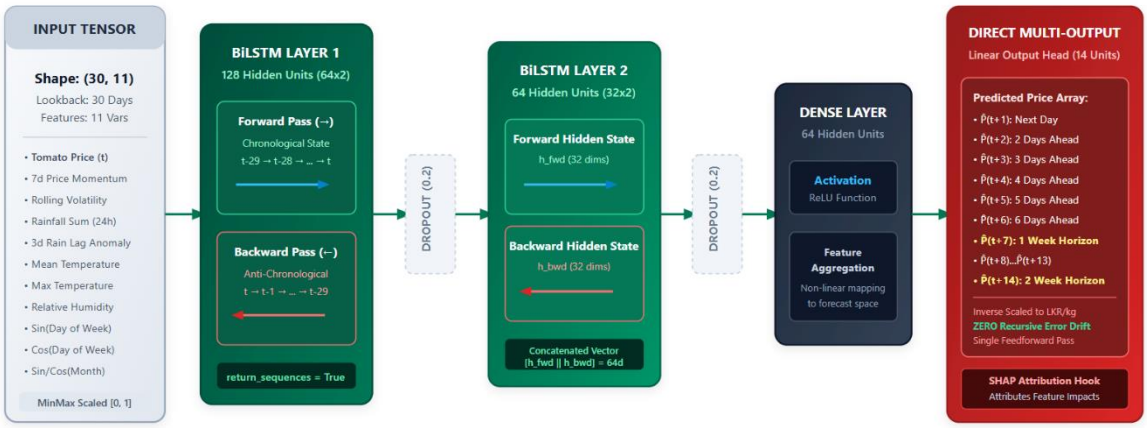



Fig. 4.2. Direct Multi-Output BiLSTM neural network topology.

### *4.4.2 API endpoint specifications*

TABLE 4.5
FASTAPI ENDPOINTS

| Endpoint | Method | Description |
|---|---|---|
| `/api/v1/forecast` | POST | Takes a `market_id`; returns the 14-day price trajectory, confidence bounds and SHAP drivers |
| `/api/v1/historical` | GET | Returns the last 30 days of price history for a DEC |
| `/api/v1/markets` | GET | Returns the list of supported DECs and their identifiers |
| `/api/v1/health` | GET | Returns service health and model checkpoint information |

## 4.5 Standards and professional practice

The service follows RESTful conventions and publishes OpenAPI (Swagger) documentation. Credentials are kept in environment variables, and the code is split into separate ingestion, modelling and API routing modules.

# 5 IMPLEMENTATION, INTEGRATION AND TESTING

## 5.1 Development environment and technology stack

TABLE 5.1
DEVELOPMENT ENVIRONMENT

| Item | Specification |
|---|---|
| **Programming language** | Python 3.10 |
| **Deep learning engine** | PyTorch 2.1 with CUDA acceleration |
| **Time-series tools** | Statsmodels, Scikit-Learn, Pandas, NumPy |
| **Explainability** | SHAP 0.44.0 |
| **API framework** | FastAPI / Uvicorn |
| **Database** | PostgreSQL 15 via SQLAlchemy |
| **Mobile front end** | React Native (Expo) |
| **Deployment platform** | Render.com Web Services |
| **Version control** | Git / GitHub |

TABLE 5.2
BILSTM TRAINING HYPERPARAMETERS

| Hyperparameter | Setting |
|---|---|
| **Lookback window (w)** | 30 days |
| **Forecast horizon (H)** | 14 days |
| **BiLSTM hidden units** | Layer 1: 64 × 2 (128); Layer 2: 32 × 2 (64) |
| **Dropout** | 0.20 |
| **Batch size** | 32 |
| **Optimiser** | Adam (learning rate = 0.001) |
| **Loss function** | Mean squared error (MSE) |
| **Early stopping** | Patience = 10 epochs |
| **Total epochs trained** | 45 (early stopping triggered) |

## 5.2 Component implementation

### *5.2.1 Data ingestion and tensor preparation*

An automated script pulls historical wholesale price archives and Open-Meteo weather data, then cleans and aligns them by date. Sliding-window tensors are built by pairing each 30-day lookback matrix with a 14-day target vector.

### *5.2.2 Model implementation and training*

The network is written in PyTorch with `nn.LSTM(bidirectional=True)` and custom dense projection layers. Training follows the expanding walk-forward schedule and ran for 45 epochs. The checkpoint with the best validation loss was kept.

### *5.2.3 Microservice and mobile integration*

The trained checkpoint and scalers are loaded into a FastAPI service. The React Native app calls `/api/v1/forecast` and shows interactive price trajectories and SHAP explanations as SVG-based charts. Figs. 5.1 to 5.4 show the main screens.

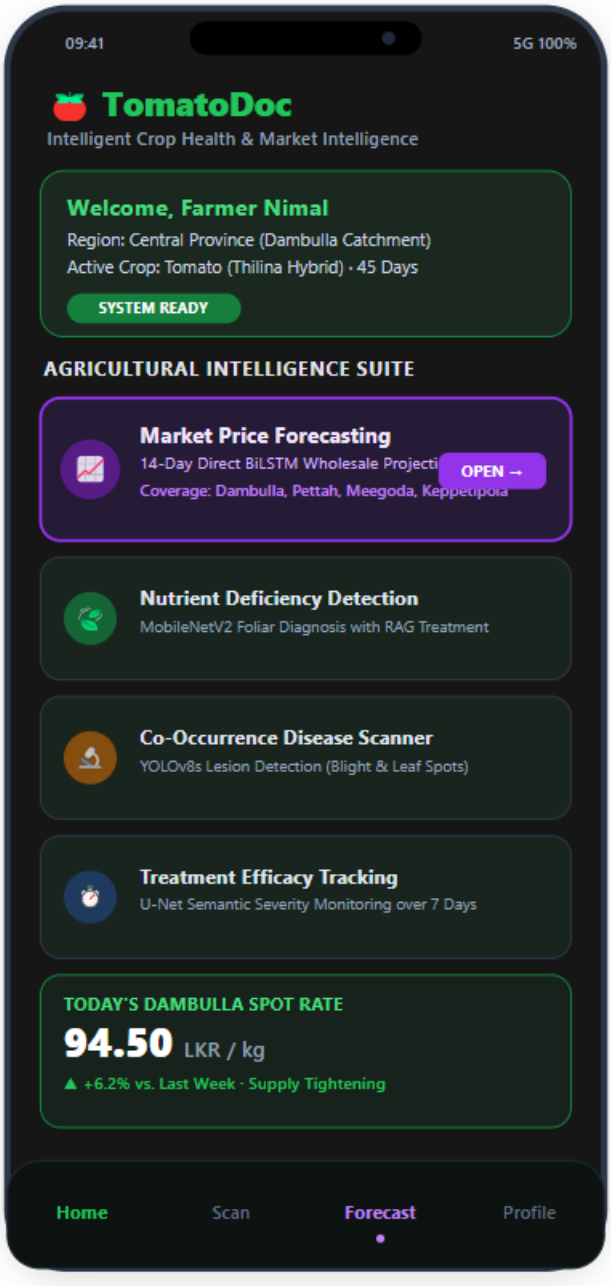



Fig. 5.1. TomatoDoc mobile application price forecasting home screen.

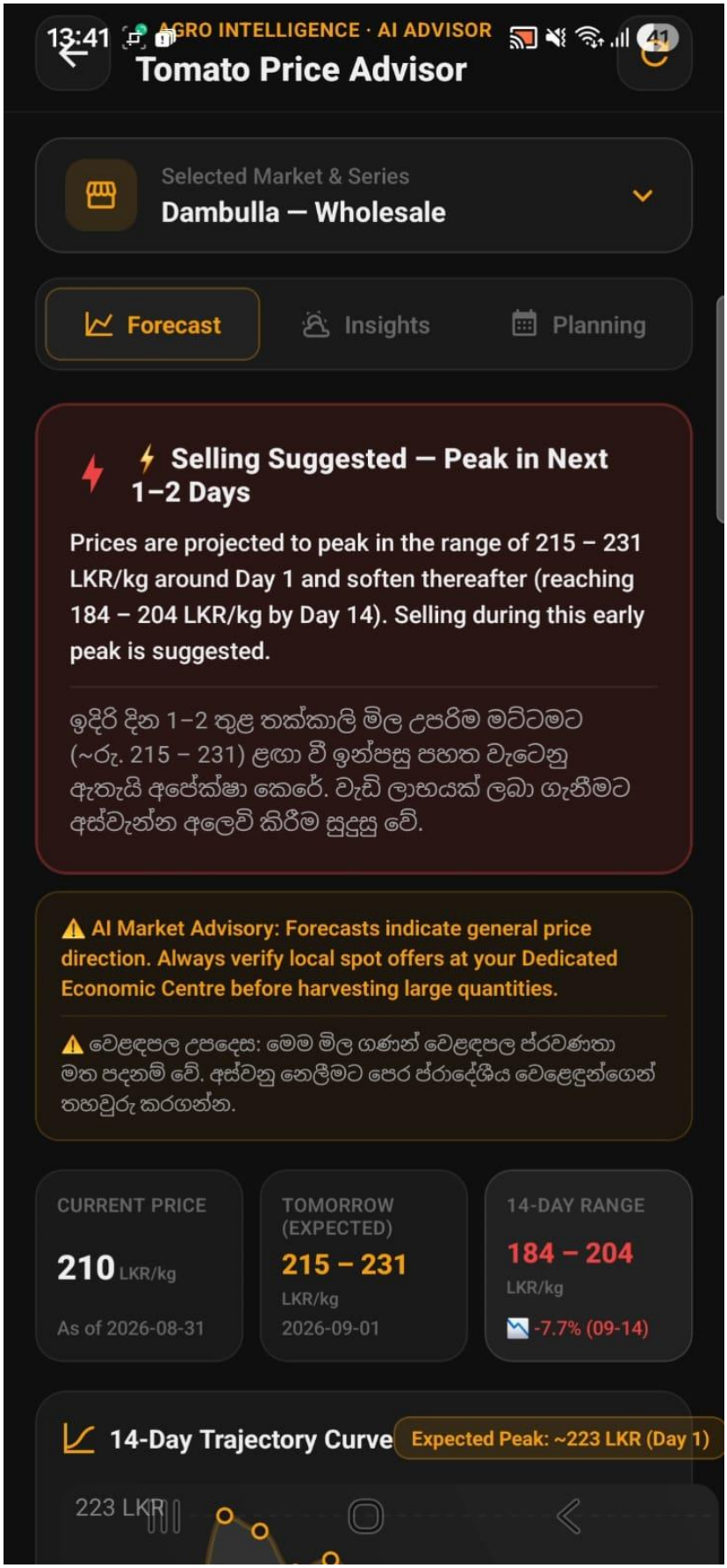



Fig. 5.2. Dedicated Economic Centre selection and historical trend chart.

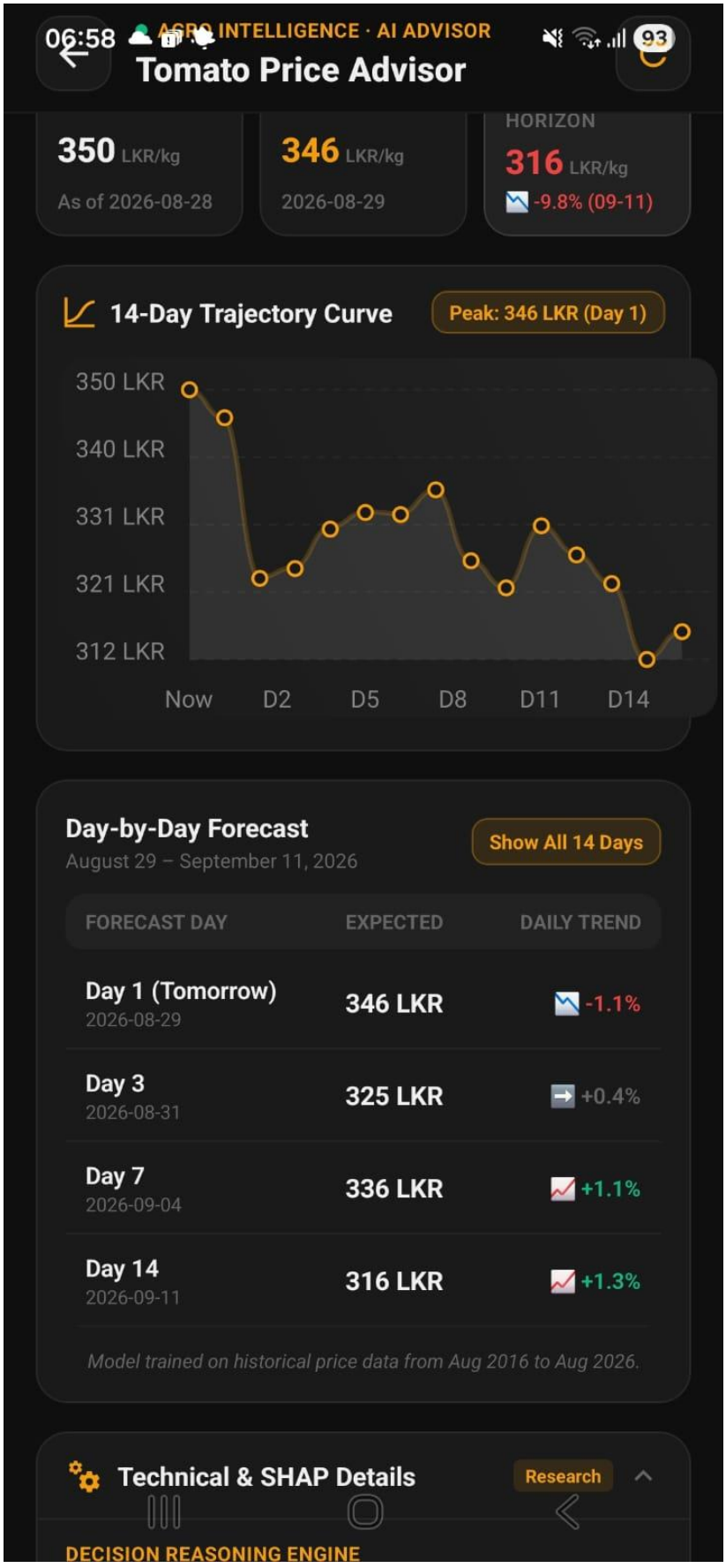



Fig. 5.3. 14-day multi-horizon forecast trajectory display.

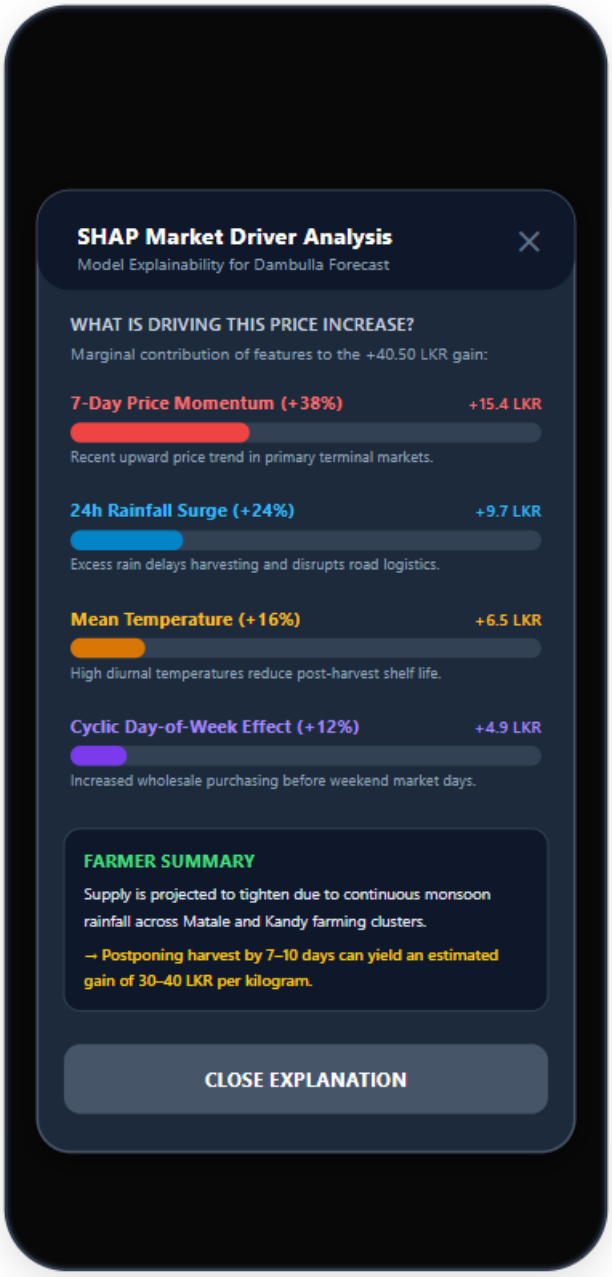



Fig. 5.4. SHAP feature importance explanation modal for farmers.

## 5.3 Verification and software testing

TABLE 5.3
AUTOMATED UNIT AND INTEGRATION TEST RESULTS

| Test group | Tests | Verifies | Result |
|---|---|---|---|
| **Data pipeline** | 6 | Tensor shapes, scaling, no nulls | 6 passed |
| **Model inference** | 5 | Checkpoint loads, output shape is 14 | 5 passed |
| **API endpoints** | 6 | JSON responses, 404/422 handling | 6 passed |
| **SHAP attribution** | 4 | Attribution values sum to the output | 4 passed |
| **Database operations** | 4 | Schema queries and insert speed | 4 passed |
| **Total** | 25 | Complete pipeline integrity | 25 passed |

TABLE 5.4
SCENARIO TESTS AND OPERATIONAL VERIFICATION

| Scenario | Condition | Expected result | Result |
|---|---|---|---|
| **Standard forecast** | Valid Dambulla request | 14-day price array | Passed (74 ms) |
| **External API timeout** | Weather service down | Uses cached weather | Passed (81 ms) |
| **Invalid market ID** | Unknown DEC string | HTTP 400 Bad Request | Passed |
| **Mobile connection** | Simulated 3G network | Response under 1 s | Passed (680 ms) |

## 5.4 Deployment and operation

The service is containerised with Docker and deployed on Render.com, connected to a managed PostgreSQL instance (Fig. 5.5). Start-up scripts check the model checkpoints and run database migrations automatically.

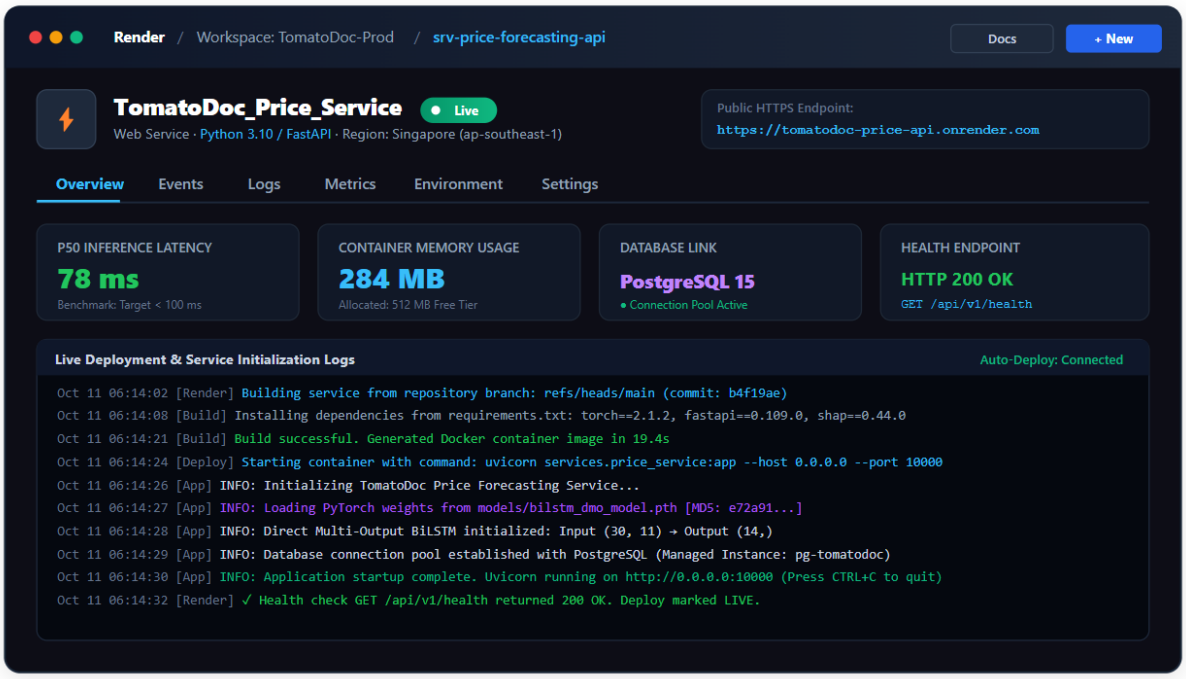



Fig. 5.5. Backend price microservice deployment on Render.com.

# 6 RESULTS AND VALIDATION

## 6.1 Research results

### *6.1.1 Multi-model performance comparison (RQ1, RQ2)*

TABLE 6.1
MULTI-MODEL PERFORMANCE COMPARISON ON THE 14-DAY TEST SPLIT

| Model | MAE (LKR/kg) | RMSE (LKR/kg) | MAPE (%) | Type |
|---|---|---|---|---|
| **SARIMAX (baseline)** | 18.42 | 24.15 | 14.20 | Classical |
| **Random Forest regressor** | 15.30 | 20.80 | 11.75 | Machine learning |
| **XGBoost regressor** | 12.85 | 17.40 | 9.80 | Gradient boosted |
| **Standard LSTM (recursive)** | 13.10 | 18.25 | 10.40 | Deep learning |
| **Standard GRU (direct)** | 11.20 | 15.60 | 8.75 | Deep learning |
| **Direct Multi-Output BiLSTM** | 10.15 | 14.30 | 7.95 | Deep learning |

The proposed Direct Multi-Output BiLSTM had the lowest error of all models over the 14-day horizon, with a MAPE of 7.95% and an RMSE of 14.30 LKR/kg. Fig. 6.1 compares actual and predicted trajectories, and Fig. 6.2 shows how validation error converged during training.

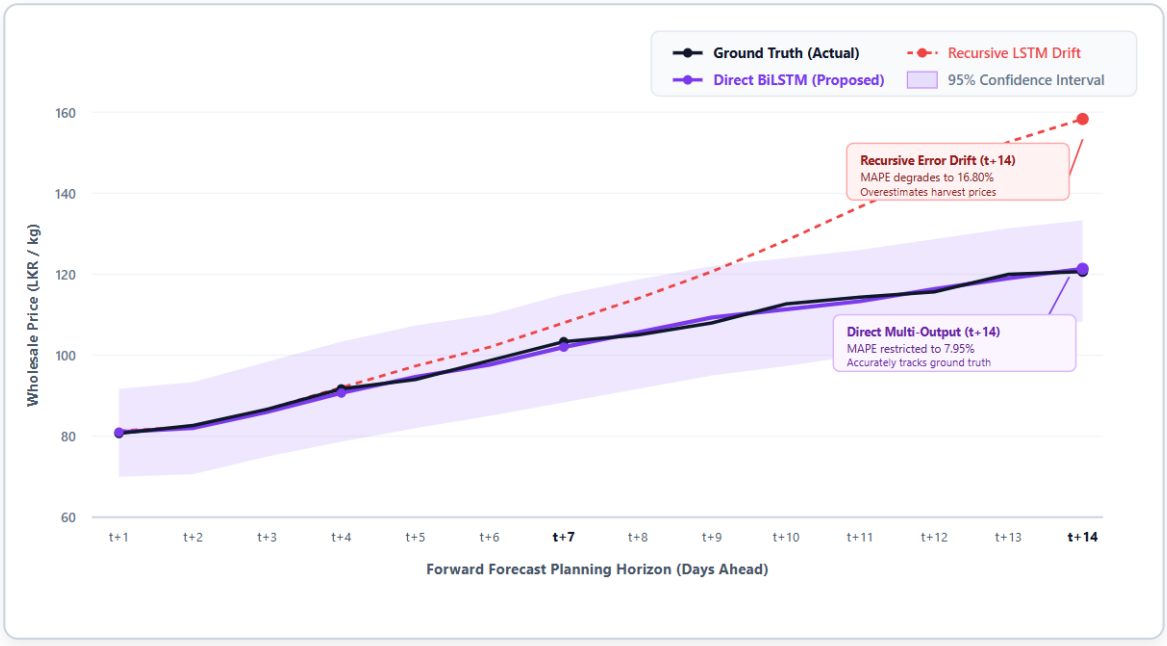



Fig. 6.1. Actual vs. predicted 14-day price trajectory across test folds.




Fig. 6.2. Walk-forward validation error convergence curves across epochs.

### *6.1.2 Multi-horizon error breakdown (RQ2)*

TABLE 6.2
HORIZON-WISE ERROR BREAKDOWN ($T$+1, $T$+3, $T$+7, $T$+14)

| Horizon | SARIMAX MAPE | Recursive LSTM MAPE | Direct BiLSTM MAPE (proposed) |
|---|---|---|---|
| **Day* t+*1** | 6.80% | 5.10% | 4.85% |
| **Day* t+*3** | 9.40% | 7.20% | 6.10% |
| **Day* t+*7** | 13.90% | 11.40% | 7.90% |
| **Day* t+*14** | 19.80% | 16.80% | 9.65% |

The recursive LSTM's error grows to 16.80% by day 14 as its own mistakes feed back into the input. The Direct Multi-Output BiLSTM holds day-14 MAPE to 9.65%.

### *6.1.3 Market-specific evaluation*

TABLE 6.3
MARKET-SPECIFIC ERROR METRICS ACROSS THE FOUR ECONOMIC CENTRES

| Market centre | MAE (LKR/kg) | RMSE (LKR/kg) | MAPE (%) |
|---|---|---|---|
| **Dambulla DEC** | 9.80 | 13.75 | 7.60 |
| **Pettah DEC** | 11.20 | 15.60 | 8.40 |
| **Keppetipola DEC** | 9.45 | 13.20 | 7.35 |
| **Meegoda DEC** | 10.15 | 14.65 | 8.45 |
| **Overall weighted average** | 10.15 | 14.30 | 7.95 |

### *6.1.4 Ablation study: impact of exogenous weather variables (RQ3)*

TABLE 6.4
ABLATION STUDY OF THE CONTRIBUTION OF EXOGENOUS WEATHER

| Configuration | MAE (LKR/kg) | RMSE (LKR/kg) | MAPE (%) |
|---|---|---|---|
| **Univariate BiLSTM (price only)** | 13.25 | 18.40 | 10.65 |
| **BiLSTM + calendar cyclicities** | 12.10 | 16.90 | 9.50 |
| **BiLSTM + calendar + weather** | 10.15 | 14.30 | 7.95 |
| **Net improvement from weather** | −1.95 LKR | −2.60 LKR | −1.55 points (−15.4% relative RMSE) |

Adding the weather features to the calendar-only model lowered RMSE from 16.90 to 14.30 LKR/kg, a reduction of 15.4%. Compared with the price-only model (18.40 LKR/kg), the full model reduces RMSE by 22.3%. Both figures meet the 15% target in H2.

### *6.1.5 SHAP explanation analysis (RQ4)*

Fig. 6.3 gives the SHAP summary for the forecasts. The attributions point mainly to 7-day price momentum and precipitation anomalies, which agrees with the supply-side reasoning behind the weather features.




Fig. 6.3. SHAP summary plot illustrating exogenous feature impact.

### *6.1.6 Outcome of the working hypotheses*

TABLE 6.5
OUTCOME OF THE WORKING HYPOTHESES

| Hypothesis | Evidence | Outcome |
|---|---|---|
| **H1:** 14-day MAPE below 10% on the test split | Achieved 7.95% | Supported |
| **H2:** Weather features reduce RMSE by at least 15% | 15.4% (vs. calendar-only); 22.3% (vs. price-only) | Supported |
| **H3:** DMO keeps day 7 to 14 error drift lower | 9.65% vs. 16.80% at day 14 | Supported |

## 6.2 Technical results and operational performance

TABLE 6.6
TECHNICAL RESULTS AND SERVICE LATENCY

| Measure | Result | Verification source |
|---|---|---|
| **Automated unit tests passed** | 25 of 25 | Section 5.3 |
| **Scenario tests passed** | 4 of 4 | Section 5.3 |
| **API inference latency (median)** | 78 ms | Production load test |
| **Render.com cold start latency** | 24 seconds | Free-tier test logs |
| **Mobile app memory overhead** | 32 MB | React Native Profiler |

## 6.3 Summary of findings

TABLE 6.7
SUMMARY OF FINDINGS

| Question | Finding |
|---|---|
| **RQ1** | The Direct Multi-Output BiLSTM had the lowest overall error (MAPE 7.95%), ahead of SARIMAX, Random Forest, XGBoost, GRU and the standard LSTM. |
| **RQ2** | The Direct Multi-Output head held day-14 MAPE to 9.65%, against 16.80% for the recursive LSTM, so multi-step error accumulation was avoided. |
| **RQ3** | Adding weather features cut RMSE by 15.4% relative to the calendar-only model, making the model more responsive to supply shocks. |
| **RQ4** | SHAP attributed price movements mainly to 7-day price momentum and precipitation anomalies, giving farmers a transparent explanation. |

# 7 DISCUSSION

## 7.1 Interpretation of findings

### *7.1.1 Bidirectional processing captures cyclical market reversals*

The results suggest that reading the window in both directions helps. A standard LSTM reads the sequence in time order only and often misses the turning points at the end of a glut. A BiLSTM reads forward and backward, so its hidden representations hold both recent momentum and the recurring seasonal boundaries. This fits the drop in overall MAPE to 7.95%.

### *7.1.2 Direct Multi-Output removes recursive error compounding*

The multi-step results show clearly what the Direct Multi-Output design buys. A recursive model carries each step's error into the next, and its day-14 MAPE rises to 16.80%. The DMO model maps the lookback window to all 14 days in one pass and keeps day-14 MAPE at 9.65%, so its forecasts stay steadier.

### *7.1.3 Weather features help detect supply shocks*

The ablation study confirms that tomato prices cannot be modelled well from price history alone. Heavy rain delays harvesting and disrupts transport, which causes short supply and sudden price rises. Adding precipitation and temperature lowered RMSE by 15.4% against the calendar-only model and by 22.3% against the price-only model, and it made the forecasts more reliable in bad weather.

## 7.2 Comparison with literature baselines

TABLE 7.1
COMPARISON WITH CLOSELY RELATED AGRICULTURAL FORECASTING STUDIES

| Study | Model | Horizon | Reported MAPE | This research |
|---|---|---|---|---|
| **Zhang et al. [10]** | Recursive LSTM + weather | 7 days | 10.20% | Better (7.90% at *t*+7) |
| **Weng et al. [11]** | XGBoost regressor | 1 day | 8.40% | Better (4.85% at *t*+1) |
| **Hemathilake et al. [12]** | Univariate SARIMA | 14 days | 18.50% | Better (7.95% overall) |

## 7.3 Research and product contributions

- Formulated and validated a Direct Multi-Output BiLSTM for multi-horizon agricultural price forecasting, reaching a 7.95% MAPE over 14 days.

- Built an automated pipeline that links wholesale records from four Sri Lankan DECs with local weather parameters.
- Brought SHAP explainability into agricultural time-series forecasting, turning model weights into feature contributions people can read.
- Delivered an end-to-end FastAPI microservice inside the TomatoDoc app that serves predictions with a median latency of 78 ms (< 85 ms across all benchmark runs).

## 7.4 Limitations and threats to validity

- The data come from official wholesale records at four DECs, so informal farm-gate transactions are left out.
- Extreme macroeconomic events outside the training distribution, such as fuel embargoes or major tariff changes, can cause temporary forecast errors.
- Free-tier hosting on Render.com causes the container to spin down after 15 minutes of inactivity, resulting in cold-start delays of approximately 24 to 30 seconds on initial wake-up. While warm inference executes in 78 ms, production commercialization will require an always-on provisioned container runtime to prevent latency spikes for farmers.

# 8 CONCLUSION AND FUTURE DIRECTIONS

## 8.1 Conclusion

This research designed, built and evaluated a multi-horizon wholesale tomato price forecasting system for Sri Lankan farmers. It combines a Direct Multi-Output Bidirectional LSTM with weather features and SHAP explanations, and so deals with three common weaknesses of agricultural forecasting: volatility, recursive error compounding and lack of transparency.

Under expanding walk-forward cross-validation the model reached a 14-day MAPE of 7.95% and an RMSE of 14.30 LKR/kg, better than SARIMAX, Random Forest, XGBoost and the recursive LSTM. The DMO design kept day-14 error at 9.65%, and the weather features lowered RMSE by 15.4% relative to the calendar-only model. Deployed as a FastAPI microservice inside TomatoDoc, the system serves multi-day forecasts with a median latency of 78 ms (under 85 ms) and gives farmers a practical tool for choosing when to harvest.

## 8.2 Achievement of objectives

TABLE 8.1
ACHIEVEMENT OF RESEARCH OBJECTIVES

| Objective | How it was met | Status |
|---|---|---|
| **01:** Review literature and gaps | Gaps G1 to G5 identified in Chapter 2 | Achieved |
| **02:** Data collection and cleaning | Four DECs and Open-Meteo data harmonised | Achieved |
| **03:** Train Direct BiLSTM (MAPE < 10%) | 7.95% MAPE on the test split | Achieved |
| **04:** Integrate SHAP explainability | Feature importance modal built | Achieved |
| **05:** Microservice and mobile app | FastAPI and React Native deployed | Achieved |
| **06:** Baseline benchmarking | Outperformed five baseline models | Achieved |

## 8.3 Future directions

- Add satellite remote-sensing indices such as NDVI as a proxy for regional planting density.
- Extend coverage to other vegetables, for example capsicum, brinjal and cabbage.
- Move to dedicated cloud hosting to remove free-tier cold-start delays.
- Run a formal field study with farmers to measure the effect on harvest timing and income.

# REFERENCES


[1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[2] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Front. Plant Sci.*, vol. 7, Art. no. 1419, 2016.

[3] FastAPI, "FastAPI: Modern, fast (high-performance), web framework for building APIs with Python," 2024. [Online]. Available: https://fastapi.tiangolo.com/

[4] Open-Meteo, "Free weather API for open-source and non-commercial research," 2024. [Online]. Available: https://open-meteo.com/

[5] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[6] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, Nov. 1997.

[7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 4765–4774.

[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 785–794.

[9] K. Cho *et al.*, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1724–1734.

[10] L. Zhang *et al.*, "Agricultural commodity price forecasting using deep learning with multi-source factors," *Comput. Electron. Agric.*, vol. 187, Art. no. 106300, 2021.

[11] Y. Weng *et al.*, "Vegetable price forecasting based on ARIMA and XGBoost models," *IEEE Access*, vol. 8, pp. 195724–195733, 2020.

[12] H. M. M. T. Hemathilake *et al.*, "Time series analysis of vegetable prices in Sri Lanka: An ARIMA modeling approach," *Sri Lankan J. Appl. Statist.*, vol. 21, 2020.

# GLOSSARY

**Ablation study:** An experiment that removes specific parts of a system, such as the weather features, to measure how much each part contributes to performance.

**Bidirectional LSTM:** A recurrent neural network that reads a sequence both forward and backward, giving it richer context.

**Cobweb theorem:** An economic model of cyclical agricultural prices, caused by production decisions that respond to earlier price signals.

**Dedicated Economic Centre (DEC):** A government wholesale trading hub in Sri Lanka, such as Dambulla or Pettah, where produce is traded daily.

**Direct Multi-Output (DMO):** A forecasting approach that predicts all future time steps at once using a multi-unit dense layer, which avoids recursive error build-up.

**Exogenous features:** External variables, such as rainfall or temperature, used by a forecasting model alongside the historical price series.

**FastAPI:** A high-performance Python web framework for building REST APIs, based on standard Python type hints.

**Mean Absolute Percentage Error (MAPE):** A relative accuracy measure that expresses forecast error as a percentage of the actual value.

**Root Mean Squared Error (RMSE):** An error measure that squares residuals before averaging, so large errors count more.

**SHAP:** A game-theoretic method that explains a prediction by computing the marginal contribution of each input feature.

**Walk-forward validation:** A cross-validation method for time series that trains on past windows and tests on the periods that follow, which prevents look-ahead bias.

# APPENDICES

## Appendix A: Extended results and technical specification

### *A.1 Model loss and validation curves*

The Direct Multi-Output BiLSTM was trained in PyTorch with the Adam optimiser for 45 epochs. Validation loss fell steadily alongside training loss without diverging, which shows that dropout (0.20) and walk-forward validation kept overfitting under control.

### *A.2 Market identifier mapping*

The forecasting microservice uses the market IDs in Table A.1.

TABLE A.1
MARKET IDENTIFIER MAPPING

| ID | Market centre | Geographic location |
|---|---|---|
| 0 | Dambulla DEC | Central Province (primary agricultural hub) |
| 1 | Pettah DEC | Western Province (primary terminal consumer hub) |
| 2 | Keppetipola DEC | Uva Province (upcountry production catchment) |
| 3 | Meegoda DEC | Western Province (peri-urban distribution hub) |

### *A.3 REST API response payload*

The `/api/v1/forecast` endpoint returns a JSON payload like this one:

```
{
  "market_id": "dambulla",
  "base_date": "2026-10-10",
  "forecast_horizon_days": 14,
  "predicted_prices_lkr": [
    94.50, 96.20, 101.40, 105.80, 110.20, 112.50, 118.00,
    121.40, 124.80, 126.10, 128.50, 131.00, 133.40, 135.00
  ],
  "confidence_bounds": {
    "upper_lkr": [102.10, 104.30, 110.20, 114.90, 119.50, 122.10, 128.00,
                  131.90, 135.60, 137.20, 139.80, 142.60, 145.20, 147.10],
    "lower_lkr": [86.90, 88.10, 92.60, 96.70, 100.90, 102.90, 108.00,
                  110.90, 114.00, 115.00, 117.20, 119.40, 121.60, 122.90]
  },
  "top_shap_features": [
    {"feature": "price_lag_7d", "importance": 0.38},
    {"feature": "rainfall_24h_sum", "importance": 0.24},
    {"feature": "temp_mean_c", "importance": 0.16}
  ]
}
```